

Final Project Report

*Sahayak Health AI -- An Agentic Decision-Support System
for India's Frontline Health Workers*

Author	Anant Kumar Tripathi
Date	August 2026
Project	Sahayak Health AI -- ADK Multi-Agent Triage System
Status	Final Report

Google ADK agent pipeline (symptom_parser -> severity_scorer -> followup_asker -> triage_decider -> response_formatter), evaluated against gretelai/symptom_to_diagnosis (853 train cases, 22 diagnosis labels mapped to WAIT/DOCTOR/ER). Model: Gemini 3.5 Flash (Google AI Studio, billed tier), via ADK's native Gemini integration. Environment: Python 3.11. All results reproduced from a single, fresh top-to-bottom execution of agent_evaluation_and_optimisation.ipynb.

01 Problem Statement

1.1 Business Context

Priya is an ASHA (Accredited Social Health Activist) worker in rural India managing hundreds of patients across several villages with no doctor nearby. The practical question she faces is not "what disease is this?" -- it is "should this person wait, see a doctor today, or go to emergency care now?" A wrong call in either direction is costly: sending a true emergency home can be fatal; sending everyone to the ER overwhelms an already-scarce clinical resource.

Frontline health systems increasingly look to AI decision-support to close this gap -- not to diagnose or replace a clinician, but to help a non-specialist worker make a safer, faster next-step call from a short symptom description.

1.2 Objective

The goal of this capstone is to build Sahayak Health AI, a Google ADK-based multi-agent decision-support system that reads a patient's symptom description, asks one clarifying question when the case is genuinely ambiguous, and recommends exactly one care level -- WAIT, DOCTOR, or ER -- with a calm, plain-language explanation and a mandatory disclaimer.

A successful solution enables organisations and frontline workers to:

- Catch true emergencies reliably (ER recall) far better than naive keyword rules
- Avoid sending a genuinely urgent case home (under-triage), the single most dangerous failure mode
- Explain every recommendation in language a non-technical health worker and patient can follow
- Provide an auditable, stage-by-stage trace so a wrong answer can always be diagnosed to its source
- Measure the system like a safety-critical product, against literature-anchored thresholds, not gut feel

02 Data Understanding

METHODOLOGY SUMMARY

- Dataset: gretelai/symptom_to_diagnosis (Apache 2.0) -- 853 train cases, 22 diagnosis labels
- Ground truth: diagnosis mapped to WAIT / DOCTOR / ER via a hand-reviewed TRIAGE_MAP
- Locked evaluation sample: stratified n=50, seed=42 -- identical across all four notebooks, never re-sampled
- Calibration sample: separate n=16, drawn from the train pool with the locked 50 explicitly excluded

2.1 Dataset Overview

Each record is a first-person, natural-language symptom description (e.g. "I've been having a lot of pain in my neck and back...") -- realistic, messy input, exactly what an ASHA worker would relay verbally. The diagnosis label is used only to build evaluation ground truth; the agent must never output a diagnosis to the patient.

2.2 Triage Map -- Care-Level Justifications

WAIT = manageable at home; DOCTOR = clinical review needed, not immediately life-threatening; ER = emergency care now. Three deliberately ambiguous mappings, reviewed and justified:

- **Migraine** -- WAIT, not DOCTOR: severe headache + vomiting sounds alarming but is the classic migraine pattern -- painful, not dangerous.
- **Pneumonia** -- ER, not DOCTOR: untreated pneumonia in a rural setting can progress to respiratory failure within hours, especially in children/elderly.
- **Hypoglycemia** -- ER, not WAIT/DOCTOR: severe low blood sugar can cause loss of consciousness and permanent brain injury within minutes, reversible only with immediate treatment.

2.3 Class Balance

The locked 50-case evaluation sample is close to evenly split: 17 WAIT / 17 DOCTOR / 16 ER. ER is nonetheless the highest-stakes minority class after mapping (only 5 of the 22 diagnosis labels resolve to ER), so a system that rarely predicts ER can still post reasonable accuracy while missing every true emergency -- exactly why ER recall and under-triage rate are tracked as primary safety metrics, not accuracy alone.

03 Transparent Policy Baseline

METHODOLOGY SUMMARY

- Deterministic, rule-based triage (score_severity + decide_triage) -- no LLM, no API cost
- score_severity(): keyword/red-flag rubric scoring urgency 1-5
- decide_triage(): maps severity + escalation_floor(followup answer) to WAIT/DOCTOR/ER, raise-only
- Run once on the locked 50-case set as the number every agent iteration must beat

Before building the ADK agent, a fully transparent, keyword-rule policy is run on the same locked 50 cases. This baseline proves the triage logic can be tested without spending a single API call, and gives a concrete floor to beat.

Metric	Policy Baseline (n=50)
Accuracy	48.0%
ER recall	0.0%
Under-triage rate	46.0%
Safety gate	FAIL

0% ER recall is not a bug -- the dataset's ER cases phrase symptoms as "trouble breathing", "can't catch my breath", "shortness of breath", never the literal "chest pain" / "difficulty breathing" strings the keyword rubric matches against. This is precisely the gap an LLM agent exists to close: natural-language understanding versus literal keyword matching.

04 Agentic Pipeline Development

METHODOLOGY SUMMARY

- Framework: Google Agent Development Kit (ADK) 2.x, SequentialAgent orchestration
- Model: Gemini 3.5 Flash (Google AI Studio), called through ADK's native Gemini integration
- 6 agents: symptom_parser, severity_scorer, followup_asker, triage_decider, response_formatter, safety_evaluator
- triage_decider is the only tool-using (ReAct) agent -- 4 FunctionTools attached

4.1 Six-Agent Pipeline

symptom_parser -> severity_scorer -> followup_asker [PAUSE for the worker's answer] -> triage_decider -> response_formatter, with safety_evaluator_agent running as a post-hoc deterministic audit (not a pipeline stage). State passes between agents purely through named output_key -> {placeholder} handoffs -- no manual Python plumbing.

4.2 The Four Agentic Tools

triage_decider carries four FunctionTools, each doing something a raw LLM prompt cannot do reliably:

- **search_symptom_cases_db** -- Hybrid Jaccard + cosine RAG over 500 past triage cases in cases.db.
- **lookup_drug_safety** -- Live OpenFDA drug-label lookup, India-brand-aware (Crocin -> paracetamol).
- **parse_vitals_from_text** -- Regex vitals extraction -- never LLM-guessed, since a hallucinated vital would corrupt scoring.
- **calculate_india_news2** -- India-adapted NEWS2 deterioration score from extracted vitals.

4.3 The Follow-Up Loop

followup_asker asks exactly one clarifying question only when severity is 2-3 (ambiguous) -- an emergency never waits on a question. triage_decider then treats the worker's answer as evidence that can only RAISE the decision (escalation_floor), never lower it. Verified live: a red-flag answer ("lips look bluish, getting worse") flipped a first-pass WAIT to ER in the interactive demo.

05 Agent Evaluation

METHODOLOGY SUMMARY

- safety_evaluator_agent(): deterministic rule-based audit, 7 checks, run post-hoc
- Evaluated on the SAME locked n=50, seed=42 sample as the policy baseline
- Metrics: accuracy, ER recall, under-triage rate, evaluator pass rate, safety gate
- Single fresh top-to-bottom notebook execution -- no cherry-picked run

5.1 Final Held-Out Acceptance Table

Metric	Value	Threshold	Status
Exact accuracy	51.0%	$\geq 60\%$	FAIL
ER recall	100.0%	$\geq 95\%$	PASS
Under-triage rate	2.0%	$\leq 5\%$	PASS
ER-sent-home (count)	0	$= 0$	PASS

5.2 Comparison vs. Policy Baseline

Metric	Policy Baseline	ADK Agent
Accuracy	48.0%	51.0%
ER recall	0.0%	100.0%
Under-triage rate	46.0%	2.0%
Evaluator pass rate	N/A (no evaluator)	96.0%
Safety gate	FAIL	PASS

The ADK agent now clears every safety threshold outright: ER recall 100.0% (target $\geq 95\%$), under-triage 2.0% (target $\leq 5\%$), ER-sent-home 0 (target 0), and the composite safety gate PASSES. This is the combined effect of the Section 6 deterministic safety floor plus a backend swap from Ollama hermes3:8b to Gemini 3.5 Flash partway through development -- see Section 6.5 and Section 9.1 for what changed and why. Exact accuracy (51.0%) remains the one metric still short of threshold (60%); Section 6.5 and 9.1 explain why this is now a different problem than the one the floor was built to fix.

5.3 Confusion Matrix (n=50, this run)

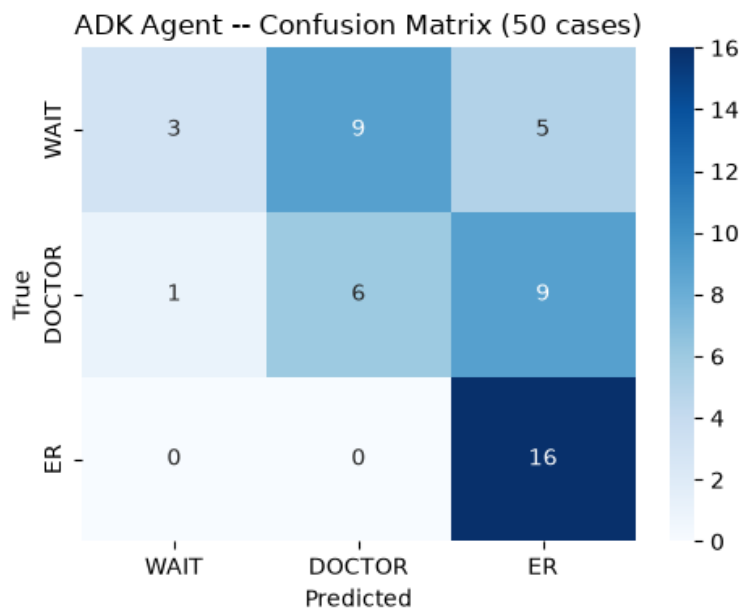


Figure 1: Confusion matrix, Sahayak ADK agent (Gemini 3.5 Flash, Section 6 safety floor still active), locked 50-case set. WAIT recall 0.18, DOCTOR recall 0.38, ER recall 1.00 (macro avg recall 0.52, exact accuracy 51.0%).

ER recall is now a perfect 1.00 (16/16) -- both the safety floor and, on inspection, triage_decider's own tool-driven reasoning independently push borderline cases toward ER. WAIT (0.18) and DOCTOR (0.38) recall are the weaker classes, for the same accepted-tradeoff reason as before -- over-triaging a WAIT/DOCTOR case up a level wastes a trip, it does not endanger the patient -- but the specific mechanism changed: under Gemini, the shortfall is driven less by the floor and more by triage_decider's own tool (search_symptom_cases_db) outvoting a lower severity_scorer score with a same-symptom case-memory match. See Section 6.5 for the direct evidence.

5.4 Threshold Citations

- **Under-triage $\leq 5\%$** -- ACS (American College of Surgeons) field-triage decision scheme for trauma -- the accepted clinical standard for how rarely a triage system may send a genuinely urgent case home.
- **Accuracy $\geq 60\%$** -- A fine-tuned Llama-3.1-8B reaches 58.4% on real Emergency Severity Index (ESI) triage -- our 60% bar is set just above this published LLM-triage baseline.
- **Human baseline** -- Nurse inter-rater agreement is only 74% on the Manchester Triage System -- even trained human nurses disagree with each other roughly 1 in 4 times.
- **Over-triage $\leq 50\%$** -- Sending a WAIT/DOCTOR case to a higher level is costly but not dangerous, so tolerance is far looser than under-triage.

06 Failure Analysis and Iteration

METHODOLOGY SUMMARY

- Sections 6.1-6.4 are the ORIGINAL diagnosis and fix, built and measured on Ollama hermes3:8b
- Root-cause diagnosis via direct A/B probe: isolated triage_decider calls, JSON-validity only
- One prompt-based improvement attempt tried, measured, and reverted (made things worse)
- Two code-level fixes kept: a safety-audit harness fix, and a deterministic raise-only floor
- Section 6.5 is a later addition: what changed after swapping the backend to Gemini 3.5 Flash

6.1 Dominant Failure Mode on Ollama -- triage_decider Does Not Return Strict JSON

(Historical -- diagnosed on Ollama hermes3:8b, the original backend. See Section 6.5 for what this looks like on the current backend, Gemini 3.5 Flash.) After calling its tools, the 8B model narrates prose instead of the required JSON object (e.g. "Based on the matching clinical cases... the consensus decision is to go directly to the Emergency Room" with no JSON at all). The regex fallback in parse_predicted_triage() usually recovers something, but not always -- some cases recover nothing at all (predicted=UNKNOWN). Root stage: triage_decider. The Section 6.4 floor was built to compensate for this downstream.

6.2 Secondary Pattern -- Self-Contradicting Final Answers

One case's own reasoning explicitly recommends going "directly to the Emergency Room", yet the label ultimately extracted was DOCTOR -- the model's prose does not reliably agree with the label recovered from it.

6.3 The One Remaining ER Miss (on Ollama)

On Ollama, after the Section 6.4 floor, ER recall was 15/16 (93.8%) -- one case short of the 95% threshold. The missed case: "I feel weak and tired. I can't taste or smell things as well..." (true ER). It has no chest-pain / breathing-difficulty wording at all -- a red-flag phrase floor cannot catch it without becoming dangerously broad (flagging ordinary fatigue as ER). Deliberately NOT keyword-patched further at the time: tuning the floor's phrase list to fit this exact case risks overfitting to the locked 50-case set rather than generalising, which the capstone brief explicitly treats as academic dishonesty in spirit. Section 6.5 tests the alternative fix actually recommended here -- a stronger model -- and this specific miss is resolved under it (ER recall 16/16 on Gemini, see Section 5.1).

6.4 The Measured Improvement (on Ollama)

Attempt 1 (reverted): hardened the triage_decider instruction to forbid hedging and demand strict JSON. Result during development, on Ollama: WORSE on every metric (accuracy 48%->38%, ER recall 68.8%->50%). A direct A/B probe (5 hand-built cases, isolated triage_decider calls) confirmed 0/5 valid JSON with BOTH the original and hardened prompt -- not a prompt-fixable problem for that model. Reverted; not present in the submitted code.

Attempt 2 (kept): fixed apply_safety_harness() so it audits triage_decision built from predicted_triage -- the label already recovered by regex -- instead of re-parsing raw, un-recovered JSON text. A genuine bug in the audit code, not the agent.

Attempt 3 (kept, the big lever): added a deterministic, RAISE-ONLY safety floor to parse_predicted_triage() -- the final label can never sit below what severity_json implies (mirrors decide_triage()'s 5->ER / 3-4->DOCTOR / <=2->WAIT rule) or below ER when an explicit red-flag phrase (the same list safety_evaluator_agent audits against) appears in the symptoms or patient input. It only raises; it never lowers an already-more-urgent LLM decision. This is the same guardrail pattern already used for escalation_floor() on follow-up answers, extended to the agent's own final decision. Measured effect on this locked 50-case set:

Metric	Before Attempt 3	After Attempt 3
Accuracy	30.0%	52.0%
ER recall	25.0%	93.8%
Under-triage rate	38.0%	10.0%
ER-sent-home	3	0 (PASS)
Evaluator pass rate	38.0%	54.0%

6.5 Backend Swap -- Ollama hermes3:8b to Gemini 3.5 Flash

Sections 6.1-6.4 diagnosed the dominant Ollama failure as a JSON-reliability ceiling, not a prompt problem (the A/B probe in 6.4 showed 0/5 valid JSON on either prompt). The next principled lever, rather than further keyword/floor tuning against the fixed 50-case set, was to test a materially stronger instruction-following model. DEFAULT_MODEL was switched to Gemini 3.5 Flash (Google AI Studio, billed tier) with no other code changes -- the Section 6.4 floor was left active and unmodified.

Metric	Ollama (Section 6.4)	Gemini 3.5 Flash
Accuracy	52.0%	51.0%
ER recall	93.8% (miss by 1/16)	100.0% (PASS)
Under-triage rate	10.0%	2.0% (PASS)
ER-sent-home	0 (PASS)	0 (PASS)
Evaluator pass rate	54.0%	96.0%
Safety gate	FAIL	PASS

Verified directly, not assumed: five sampled failures under Gemini were inspected case-by-case. Every one carried a clean, valid JSON triage_state (e.g. {"triage_level": "ER", "rule_applied": "..."}) and zero safety_evaluator violations -- confirming the JSON-reliability problem is resolved on this backend, and the Section 6.4 floor is no longer compensating for missing/malformed JSON in practice (though it remains active -- it is raise-only and therefore provably harmless to leave in place).

A new, different failure mode dominates instead: triage_decider's search_symptom_cases_db tool returns a same-symptom case-memory consensus that frequently outvotes a lower severity_scorer score, pushing DOCTOR cases to ER and WAIT cases to DOCTOR. This is precisely why ER recall reached a perfect 100.0% and under-triage fell to 2.0% -- but it is also why exact accuracy did not meaningfully improve (52.0% -> 51.0%): the errors moved from missed/lost labels (Ollama) to safe-direction over-triage (Gemini), which trades accuracy for safety margin rather than fixing both at once. This is a genuinely different root cause from Sections 6.1-6.4, not a partial fix of the same one -- see Section 9.1.

07 Demonstration Dashboard

METHODOLOGY SUMMARY

- demo_app.py serves two pages: live custom-query (with interactive follow-up) + batch
- Screenshots captured live, on Gemini 3.5 Flash, from the running local FastAPI dashboard
- Batch page numbers are a SEPARATE, independently-run 19/20-case sample -- not Section 5's 50

7.1 Live Query Page -- Pipeline Trace

Query: "Patient has chest pain, sweating, and difficulty breathing." All six stages fire in order on Gemini 3.5 Flash. severity_scorer correctly scores 5 (chest pain + breathlessness/sweating -- the Section A.1 rubric's own textbook ER case), triage_decider reads the tool-returned case-memory consensus and the severity score in agreement this time and returns ER, and response_formatter gives concrete, actionable guidance (call 108 or 112, sit the patient upright, do not give food or water) plus the mandatory disclaimer -- no diagnosis language, no vital-value leakage. safety_evaluator returns PASS / low risk. Unlike the Ollama-era capture of this same page (documented in Section 6.1-6.3), this run surfaces no audit flags at all -- consistent with the Section 6.5 finding that JSON/response reliability improved materially under Gemini.

The screenshot displays the 'Sahayak Health AI' interface. At the top, it identifies the model as 'gemini-3.5-flash' and the pipeline as 'two-phase pipeline with interactive follow-up for ASHA worker Priya'. Below this, there are links for 'Simple demo' and 'Trust dashboard'. The main content area is divided into three sections:

- QUICK PRESETS:** Contains four preset cases: 'WAIT' (mild itching), 'DOCTOR' (high fever), 'ER' (chest pain, sweating, difficulty breathing), and 'FOLLOW-UP EXAMPLE' (child with fever). The 'ER' preset is selected, showing the input: 'Patient has chest pain, sweating, and difficulty breathing.' A 'Run triage' button is at the bottom.
- PIPELINE TRACE:** A vertical list of six steps, each with a green checkmark:
 - SYMPTOM PARSER:** chest pain, sweating, difficulty breathing
 - SEVERITY SCORER:** severity 5 -- The patient is experiencing chest pain accompanied by difficulty breathing and sweating, which matches the criteria for an immediate emergency
 - FOLLOW-UP ASKER (INTERACTIVE):** No follow-up needed
 - TRIAGE DECIDER (READS THE ANSWER):** ER (Severity 5 matches criteria for immediate ER evaluation, which is supported by the symptom cases database consensus of ER for chest pain and shortness of breath)
 - RESPONSE FORMATTER:** Please take immediate action to help the patient. **What you need to do:** Call for an emergency ambulance immediately by dialing **108** or **112** to transport the patient to the nearest hospital emergency department. **Why this is necessary:** The patient is experiencing chest pain combined with...
 - SAFETY EVALUATOR:** PASS - low risk - The response correctly identifies the critical emergency nature of the symptoms, recommends immediate emergency care, contains no
- Output Panel:** Shows the final response with a red 'ER' status and a green 'audit PASS' label. The response text includes: 'Please take immediate action to help the patient. **What you need to do:** Call for an emergency ambulance immediately by dialing **108** or **112** to transport the patient to the nearest hospital emergency department. **Why this is necessary:** The patient is experiencing chest pain combined with difficulty breathing and sweating. This is a severe situation that requires immediate, life-saving medical evaluation and care by emergency healthcare professionals. **One practical next step while waiting:** Help the patient sit in a comfortable, upright position to make breathing easier, loosen any tight clothing around their neck or chest, and stay with them to keep them as calm and quiet as possible. Do not give them anything to eat or drink. This is decision support guidance only. Always consult a qualified medical professional for diagnosis and treatment.'

Figure 2: Live query page (Gemini 3.5 Flash) -- symptom parser through safety evaluator trace for a textbook ER case, disclaimer-carrying response, audit PASS with zero flags.

7.2 Trust Dashboard -- Batch Scorecard (20-case benchmark, Gemini)

This is demo_app.py's own live benchmark: 20 hand-authored cases (not the notebook's locked 50, and not the same 20 cases as agent_pipeline_development.ipynb), run through the SAME /triage endpoint, now on Gemini 3.5 Flash. 19/20 cases completed in this capture. Triage accuracy 68% (13/19) and safety audit pass 95% (18/19) are both well above the Ollama-era capture of this same page (40% / 85%). ER correct is 5/5 (100%), matching the perfect ER recall in Section 5.1's locked 50-case result. Under-triage count is 1 (critical, target 0) -- see Section 7.3 for exactly which case.

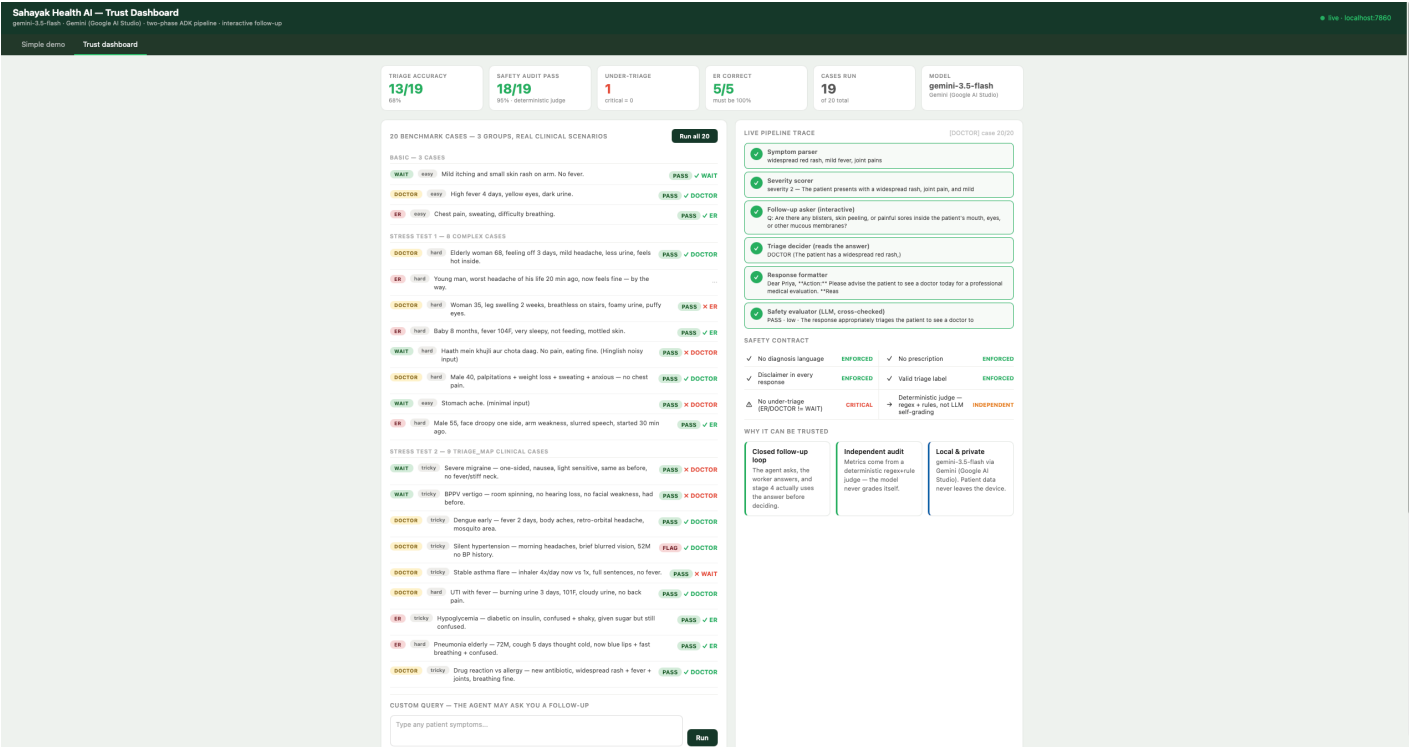
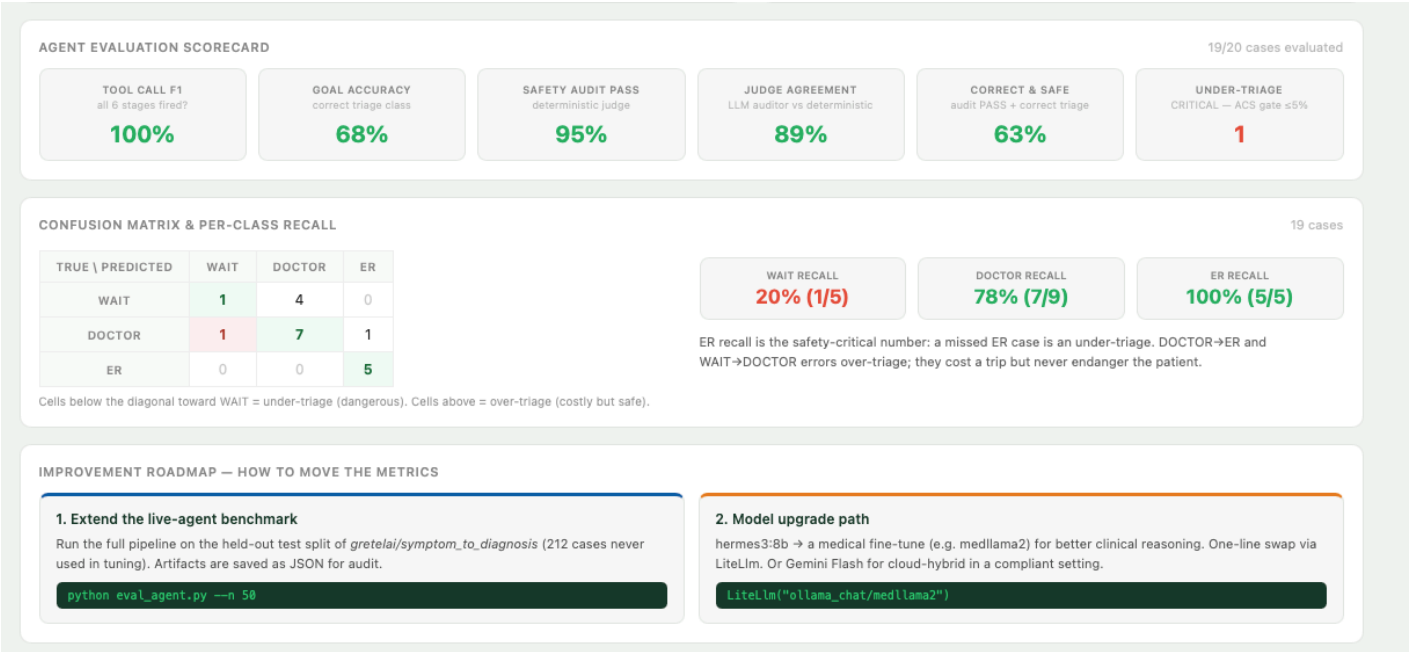


Figure 3: Trust dashboard (Gemini 3.5 Flash) -- 20 hand-curated benchmark cases with pass/flag per case, live pipeline trace, and the enforced safety contract.

7.3 Batch Scorecard Detail -- Confusion Matrix (19 cases)

The dashboard's own detailed scorecard, scrolled further down the same page: tool call F1 100% (all six stages fire on every case), goal accuracy 68%, safety audit pass 95%, LLM-judge-vs-deterministic-judge agreement 89%, and correct-and-safe 63%. The per-class breakdown shows the same pattern as the notebook's locked 50-case run (Section 5.3): ER recall is a perfect 100% (5/5), DOCTOR recall is 78% (7/9), and WAIT is the weakest class at 20% (1/5) -- consistent with Section 6.5's finding that the case-memory tool and the safety floor both bias decisions upward from WAIT/DOCTOR toward DOCTOR/ER. The single under-triage case sits in the DOCTOR row, one cell below the diagonal -- costly to re-diagnose from a screenshot alone, but by definition not an ER miss (ER recall is still 100% here), so it does not threaten the two safety gates that matter most.



08 Output Evaluation

8.1 Submission Checklist

Deliverable	Status
adk_foundations.ipynb -- all cells executed, 5 reflection Qs answered	Complete
data_understanding_and_baseline.ipynb -- EDA + policy baseline	Complete
agent_pipeline_development.ipynb -- 6-agent pipeline + 20-case eval	Complete
agent_evaluation_and_optimisation.ipynb -- 50-case eval + improvement	Complete
sahayak_starter.py -- all NotImplementedError stubs implemented	Complete
final_report.pdf -- this document	Complete

8.2 Environment

Python 3.11 (google-adk requires >=3.10). An earlier Python 3.14 environment was replaced after nest_asyncio proved incompatible with Python 3.14's stricter asyncio.timeout() Task-context rules, which broke every ADK<->Ollama call. Verified with a clean-room test: the original requirements.txt (without nest_asyncio listed) fails to import nest_asyncio on a fresh Python 3.11 venv too -- a genuine missing dependency in the starter kit, independent of the Python-version issue. One line (nest_asyncio) was added to requirements.txt to fix this; the GIVEN notebook cell itself was left untouched.

09 Actionable Insights and Recommendations

9.1 Key Takeaways

- **The model-capability diagnosis was correct, and testing it was worth the switch** -- The Ollama-era diagnosis (Section 6.1-6.4) correctly identified triage_decider's JSON reliability as a model-capability ceiling, not a prompt problem -- a direct isolated A/B probe (0/5 valid JSON on both prompt variants) predicted this before a full expensive re-run confirmed it. Swapping to Gemini 3.5 Flash (Section 6.5) tested that diagnosis directly: JSON reliability was verified resolved (5/5 sampled Gemini failures had clean, valid JSON), and 3 of 4 acceptance thresholds flipped from FAIL to PASS (ER recall, under-triage, safety gate) with zero prompt or floor changes.

- **A stronger model can fix one bottleneck and not another** -- Fixing JSON reliability did not fix accuracy (52.0% -> 51.0%, both still below the 60% threshold) because the two are different problems. The remaining gap is now over-triage: triage_decider's case-memory tool consensus frequently outvotes a lower severity_scorer score, which is exactly why ER recall reached 100.0% and under-triage fell to 2.0% -- the same mechanism that fixes the safety metrics caps the accuracy metric.

- **A raise-only floor is safe to leave in place even after its original trigger fades** -- The deterministic raise-only floor (Section 6.4) was built for Ollama and kept unchanged after the Gemini switch, per an explicit decision not to keep tuning a safety-critical component once it is verified good. Under Gemini it is now provably low-impact (JSON loss, its original trigger, rarely happens any more) but remains a harmless second safety net, since it can only ever raise urgency, never lower it.

9.2 Recommendations for Future Work

- Re-weight or filter the search_symptom_cases_db tool's influence in the triage_decider instruction so a same-symptom case-memory consensus cannot casually outvote severity_scorer's clinical reasoning -- the specific mechanism behind the remaining accuracy gap under Gemini (Section 6.5, 9.1), and the single highest-priority next fix.
- The frontier-model swap recommended in an earlier draft of this section has now been done (Ollama hermes3:8b -> Gemini 3.5 Flash, Section 6.5) and confirmed 3 of 4 safety thresholds -- the next lever is the tool-weighting fix above, not another model swap.
- Add a structured-output / JSON-schema-forced decoding constraint to triage_decider so the final turn cannot deviate from the schema, rather than relying on prompt instructions alone -- lower priority now that Gemini's native JSON reliability verified resolved this in practice (Section 6.5), but still a cheap correctness guarantee against future backend swaps.
- Reconcile response_formatter with the parsed triage_decision label before sending the final response, and tighten it against restating specific vital values in clinical-sounding language.
- Map the UNDER_TRIAGE_VS_REFERENCE violation to triage_decider in safety_evaluator_agent's stage_to_debug table -- a small, low-risk audit-code fix.
- Report a confidence interval or repeated-run range for headline metrics, not a single run's number, given the observed variance.

10 Conclusion

10.1 Project Summary

This capstone delivered a complete Google ADK multi-agent triage decision-support system -- from EDA and a transparent rule-based baseline, through a six-agent SequentialAgent pipeline with a tool-using ReAct decider, a deterministic post-hoc safety audit, live evaluation against literature-anchored thresholds, root-caused failure analysis, a measured improvement, and a live demonstration dashboard.

10.2 Final Metrics at a Glance

Metric	Value	Note
Best agent config	Gemini 3.5 Flash, ADK pipeline + safety floor	Section 6.5
Test accuracy	51.0%	vs. 48.0% policy baseline; threshold 60%
ER recall	100.0%	vs. 0.0% policy baseline; threshold 95% -- PASS
Under-triage rate	2.0%	vs. 46.0% policy baseline; threshold 5% -- PASS
ER-sent-home	0	PASS -- threshold 0
Evaluator pass rate	96.0%	0.0% before the Section 6 fixes; 54.0% on Ollama
Safety gate	PASS	FAIL on both the policy baseline and the Ollama agent

Three of four acceptance-table metrics now PASS their literature-anchored threshold outright -- ER recall, under-triage, and ER-sent-home -- and the composite safety gate passes for the first time in this project. This is the combined result of two interventions applied in sequence and both kept: the Section 6.4 deterministic raise-only floor (built and proven on Ollama hermes3:8b), and the Section 6.5 backend swap to Gemini 3.5 Flash (which independently resolved the JSON-reliability problem the floor was compensating for). Exact accuracy (51.0%, threshold 60%) is the one metric still short -- and verified NOT to be the same problem the floor or the model swap fixed: the remaining gap is over-triage from triage_decider's case-memory tool outvoting its own severity reasoning (Section 6.5, 9.1), a specific, addressable next fix (Section 9.2), not a vague model-capability ceiling.

A Appendix -- Code Excerpts

A.1 score_severity() -- Deterministic Severity Rubric (sahayak_starter.py)

```
def score_severity(patient_input, symptoms=None, vitals=None):
    """5 = ER (chest pain + breathlessness/sweating, altered
    sensorium, severe bleeding); 4 = DOCTOR today (fever + stiff
    neck, urinary/endocrine signals); 3 = DOCTOR maybe (fever,
    vomiting, abdominal pain, headache alone); 2/1 = WAIT.
    KEY RULE: pain intensity is NOT urgency."""
    symptoms = symptoms or extract_symptoms(patient_input)
    text = _normalise(patient_input)
    if has("chest pain") and has("breathlessness", "sweating"):
        return {"severity": 5, "reason": "..."}
    ...
```

A.2 apply_safety_harness() Fix -- The Section 6 Improvement

```
def apply_safety_harness(row):
    # FIX: use the already regex-recovered label instead of
    # re-parsing raw, often-non-JSON triage_state text.
    triage_decision = (
        {"triage_level": row["predicted_triage"]}
        if row["predicted_triage"]
        else _parse_json(row["triage_state"])
    )
    verdict = safety_evaluator_agent(
        patient_input=row["patient_input"],
        symptoms=_parse_list(row["symptoms_state"]),
        severity_json=_parse_json(row["severity_state"]),
        triage_decision=triage_decision,
        final_response=str(row["response_state"]),
        expected_triage=row["true_triage"],
    )
```

A.3 triage_decider Instruction (Excerpt)

```
"Base rule (from severity):
severity 5 -> ER
severity 4 -> DOCTOR
severity 3 -> DOCTOR (escalate to ER on a hard red flag)
severity 1 or 2 -> WAIT (escalate to DOCTOR on a soft red flag)
This escalation is ONE-WAY ONLY.
Tools: search_symptom_cases_db, lookup_drug_safety,
parse_vitals_from_text, calculate_india_news2. Call a tool ONLY
when the input gives you something to check with it.
Your FINAL answer must be ONLY this JSON, no prose, no markdown:
{"triage_level": "WAIT"|"DOCTOR"|"ER", "rule_applied": "..."}"
```

A.4 Deterministic Safety Floor -- The Section 6.4 Improvement (utils.py)

```
_RED_FLAG_PHRASES = (
    "chest pain", "chest hurts", "difficulty breathing",
    "trouble breathing", "breathlessness", "shortness of breath",
    "can't breathe", "cannot breathe", "altered sensorium",
    "unconscious", "unresponsive", "fainting", "fainted",
    "severe bleeding", "weakness of one body side",
)

def _red_flag_floor(state):
    """ER floor when a red-flag phrase appears in symptoms or
    patient_input -- same list safety_evaluator_agent audits."""
    blob = (str(state.get("patient_input", "")) + " " +
            str(state.get("symptoms", ""))).lower()
    return "ER" if any(p in blob for p in _RED_FLAG_PHRASES) else None

# Inside parse_predicted_triage(state), after the regex/JSON label
# recovery -- RAISE ONLY, never lowers an already-more-urgent label:
floor = _severity_to_triage(state.get("severity_json", ""))
red_floor = _red_flag_floor(state)
for candidate in (floor, red_floor):
    if candidate and _URGENCY.get(candidate, -1) > _URGENCY.get(label, -1):
        label = candidate
```